Tree-MDP: Treating Tree Search as Markov Decision Process

Abstract

The efficiency of tree search algorithms for deterministic symbolic search and symbolic planning crucially depends on the heuristic functions. Due to the difficulty of their design, there is a growing interest in implementing them with neural networks and optimizing them for a given distribution of problems and domains. Despite the success of reinforcement learning (RL) in many fields and its similarity to planning, there is generally little inspiration from RL in the design of heuristic functions. We therefore propose to formulate the tree search algorithms as a Markov Decision Process (further called Tree-MDP to prevent confusion), which considers the dynamics of the solved problem within the tree search algorithms, and where each state of the Tree-MDP corresponds to a whole partially expanded search tree, not one just node. The Tree-MDP formulation allows us to optimize heuristic functions using existing off-the-shelf RL algorithms, e.g. Proximal Policy Optimization algorithm. Relating this formulation to recently proposed ranking losses leads to a formulation blending ranking losses and RL methods, leading to a powerful new method. Experimental evaluation on problems from the IPC 2023 learning track validates our approach.

1 Introduction

Tree search algorithms, such as A* [Hart *et al.*, 1968], IDA* [Korf, 1985], and Greedy Best-First Search (GBFS) [Nilsson, 1971] that find the best path from an initial state to a goal state are among the key algorithms of Artificial Intelligence. They have been successfully applied in diverse domains, including robotic planning [Silver *et al.*, 2021] and games [Silver *et al.*, 2016], among others. The success and efficiency of these algorithms crucially depend on the quality of the heuristic functions that select which node should be expanded next.

Because designing effective heuristic functions is challenging, a growing number of works use machine learning techniques. One of the earliest approaches is optimization of a combination of existing heuristics in portfolio planning [Franco *et al.*, 2017; Moraru *et al.*, 2019; Katz *et al.*, 2018]. A more ambitious goal is to learn heuristic functions directly from states [Bhardwaj *et al.*, 2017; Ferber *et al.*, 2020; Groshev *et al.*, 2018; Toyer *et al.*, 2020; Horčík and Šír, 2024; Chen *et al.*, 2024], usually using strong function approximators like Support Vector Machines or Neural Networks.

Complementary research focuses on criteria (loss functions) optimized while learning the heuristic functions. The most popular is to optimize the cost-to-go, which is considered the optimal heuristic guaranteeing optimality of A* search algorithm and efficiency of GBFS. But estimating cost-to-go might be restrictive and does not guarantee that search will find the solution while expanding the minimal number of nodes (called strict optimal efficiency in [Chrestien *et al.*, 2023]). Therefore alternatives [Chrestien *et al.*, 2023; Piepenbrock *et al.*, 2022; Hao *et al.*, 2024] optimize efficiency of the search by forcing states on optimal trajectory to rank higher than alternatives. The rationale behind this is that the efficiency of the tree search algorithms is governed by the relative order rather than the absolute values of the heuristic function.

Since heuristic functions seem to be similar to the value and/or Q-function in reinforcement learning (RL), the scarce use of RL algorithms to optimize the forward search is surprising, especially in cases where the solved problem instances are not available. Some works [Chen and Tian, 2019; Zoph and Le, 2017; Anderson *et al.*, 2018] use RL algorithms to solve the problem without resorting to the tree search, but they strip themselves of the advantages, such as never visiting the same states and backtracking from dead ends. Closer to this work is a conversion of policy optimized by RL algorithms to tree search proposed in [Orseau *et al.*, 2018], but [Chrestien *et al.*, 2023] shows that heuristics optimizing efficiency are better.

This paper addresses a fundamental question: why have reinforcement learning (RL) algorithms not been more widely adopted for learning heuristic functions in classical planning? This is important since RL solves similar problems like sparse reward and exploration. We argue that the key obstacle is a mismatch in problem formulation. Standard RL algorithms optimize policies and value functions for an agent acting directly in the original problem space, where states represent configurations of the domain, and actions are domain oper-

Algorithm 1: Greedy Best-First Search (GBFS).

```
function GBFS(n0, heuristic)
    open_list = PriorityQueue()
    enqueue!(open_list, n0, heuristic(n0))
    close_list = Set()

    while !isempty(open_list)
        # remove node of minimum value
        n = dequeue!(open_list)

        if n in close_list
            continue # pruning
        end
        push!(close_list, n)

        if is_solution(n)
            return n
        end

        # Node expansion
        for c in children(n)
            enqueue!(open_list, c, heuristic(c))
        end
    end
    return false
end
```

ators. However, forward search algorithms operate in a different space: they incrementally build a search tree by selecting which nodes to expand. Naively applying RL to learn heuristics ignores this distinction and fails to account for the dynamics of the search process itself.

To address this, we introduce the Tree-MDP formalization, which recasts forward search as a Markov Decision Process. In Tree-MDP, a state is the current search tree (including both explored and open nodes), and an action is the selection of a node from the open list for expansion, which allows RL algorithms—particularly policy gradient and Q-learning methods—to directly optimize the node selection strategy. In [Bhardwaj *et al.*, 2017], a formulation similar to Tree-MDP is mentioned, but not carried further due to fear of high computational costs. The optimal policy in Tree-MDP naturally corresponds to a strictly efficient heuristic [Chrestien *et al.*, 2023]: one that guides the search to expand the minimal number of nodes. We further investigate how Tree-MDP relates to existing ranking-based loss functions [Chrestien *et al.*, 2023; Piepenbrock *et al.*, 2022; Hao *et al.*, 2024], providing a unified perspective on recent advances in learned heuristics. This perspective gives rise to a new, powerful method based on ideas from planning and RL. Experimental evaluation demonstrates the practical benefits and versatility of the proposed method across several benchmark domains. The proposed method, L_{po} improves the established prior art by 30% and improve domain, where most algorithms fail.

2 Problem Definition and Background

Problem definition. This paper considers a class of planning problems typically solved by heuristic search, such as greedy best-first search (GBFS) considered in this work. We assume a set of states $\mathcal{S}$, a set of actions $\mathcal{A}$, and a deterministic transition function δ. The goal, given an initial state $s_0 \in \mathcal{S}$, is to find a sequence of actions $(a_0, a_1, \ldots, a_{n-1})$, such that $s_{i+1} = \delta(s_i, a_i)$, where $(s_0, s_1, \ldots, s_n)$ is a solution path maximizing the objective function $o(s_n)$,[1] where $o\colon \mathcal{S} \to \mathbb{R}$ is a scalar function quantifying the quality of the state. We assume that, for a given s_0, the maximizing $o(s_n)$ that can be reached from s_0 is bounded by some constant. Furthermore, we emphasize that o by definition does not depend on the path from s_0 to a state s_n.

Markov Decision Process. We assume a usual definition of Markov Decision Process (MDP) as a tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \delta, R)$, where $\mathcal{S}$ is a set of states, $\mathcal{A}$ is a set of actions, $\delta\colon \mathcal{S} \times \mathcal{A} \to \mathcal{S}$ is the deterministic transition function, and $R\colon \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is the reward function. The objective in MDP is to maximize reward $\sum_{t=0}^{n-1} \gamma^t R(s_t, a_t)$, where γ is a discount factor. In episodic tasks, the horizon n is finite, and the discount factor is usually $\gamma = 1$, cf. Section 4.1. In contrast, for infinite horizon tasks, $n = +\infty$ and $0 < \gamma < 1$.

To make MDP applicable for the problem of maximizing the scalar function, o, the popular approach is to define a reward function as $R(s, a) = o(\delta(s, a)) - o(s)$ [Chen and Tian, 2019]. With that it holds for a solution path $(s_0, s_1, \ldots, s_n)$ that $\sum_{i=0}^{n-1} o(s_{i+1}) - o(s_i) = o(s_n) - o(s_0)$ and hence it depends only on s_0 and s_n. In satisfying planning and similar problems, the reward function $R(s, a)$ is non-zero only if $\delta(s, a)$ is a goal state, which makes the problem sparse and challenging for RL algorithms.

Search methods typically run in an episodic setting, since the time, space, and other resources are finite. To encourage the search to find the shortest path, we can use a discount factor $\gamma < 1$. However, this can introduce local minima as discussed in Section 4.

Reinforcement learning. RL algorithms aim to find a policy $\pi\colon \mathcal{S} \to \mathcal{A}$ maximizing the expected accumulated reward [Sutton *et al.*, 1998]. The optimal policy, denoted by π^*, can be derived either from the optimal action-value function Q^*, where $Q^*\colon \mathcal{S} \times \mathcal{A} \to \mathbb{R}$, or the optimal state-value function V^*, where $V^*\colon \mathcal{S} \to \mathbb{R}$, or alternatively, it can be learned directly by methods optimizing policy, such as currently popular proximal policy optimization (PPO) [Schulman *et al.*, 2017] methods.

The optimal value function in the episodic setting, assuming $\gamma = 1$, is equal to the negative of the cost-to-goal, i.e. to the cumulative reward the agent can receive from state s. More generally, for deterministic transitions and any discount factor γ, the optimal value function satisfies the Bellman equation

$$V^*(s) = \max_{a \in \mathcal{A}} \Big[R(s, a) + \gamma V^*\big(\delta(s, a)\big) \Big]. \qquad (1)$$

Following [Sutton *et al.*, 1998], the optimal policy param-

[1] For simplicity, we assume the episode terminates when the maximum is reached, although in practice this does not hold.

eters θ are found by maximizing

$$\mathbb{E}_{\xi \sim \pi_\theta}[G(\xi)], \tag{2}$$

where $\xi = (s_0, s_1, \ldots, s_n)$ is a trajectory sampled by policy π_θ.[2] Most methods (like PPO) assume the additive reward structure $G(\xi) = \sum_{t=0}^{n-1} \gamma^t R(s_t, a_t)$. Policy optimization methods are now the preferred choice for many tasks, such as generative AI, but value-based (or Q-learning) methods retain their popularity.

Greedy Best-First Search (GBFS). While the ideas in this work can be applied to other search algorithms, this work exclusively uses GBFS, briefly recapitulated below and in Algorithm 1. GBFS is a tree search algorithm that can be applied to any labeled transition system to find a path from the initial state s_0 to the goal state. At each iteration, the GBFS expands the most promising state, as measured by the heuristic function, h, which has not yet been expanded (these states are stored in the *open list*) and continues until a goal state is reached or time or other limits are reached. Due to its greedy nature, GBFS offers no guarantees of optimality or quality for the resulting plan. In this paper, the transition system is defined by the MDP. Depending on the objective, GBFS is either used to find a solution or to find a state maximizing an objective function o within a given limit. To improve exploration, the `dequeue!(open_list)` in line 8 is replaced by sampling from a categorical distribution with log-probabilities defined by heuristic values — a common formulation in RL.

3 Tree-MDP

This section formalizes the GBFS as an MDP problem (called Tree-MDP), which subsequently enables the use of existing RL methods, such as the currently popular PPO, to optimize heuristic functions.

Recall we assume to have a set of states $\mathcal{S}$, a set of actions $\mathcal{A}$, and the transition function δ. The goal is for given initial state $s_0 \in \mathcal{S}$ to find a sequence of actions $(a_0, a_1, \ldots, a_{n-1})$, such that $s_{i+1} = \delta(s_i, a_i)$ and s_n maximizes bounded objective function o.

As discussed above, the problem can be solved by RL methods with an appropriate reward function. But we believe (and experimental results below confirm that) that this straightforward approach might lead to inefficiency caused by (i) visiting node multiple times[3] and (ii) more importantly by the inability to backtrack from local minima or dead ends. The GBFS elegantly solves these problems by using the search-tree structure.

If RL algorithms are to be used to learn a heuristic function for forward-search algorithms, the resulting Tree-MDP needs to satisfy the Markov property. Therefore for a given $\mathcal{S}, \mathcal{A}, \delta$, objective function o, and initial state s_0, the corresponding Tree-MDP is a tuple $\tilde{\mathcal{M}} = (\tilde{\mathcal{S}}, \tilde{\mathcal{A}}, \tilde{\delta}, \tilde{R})$ defined as follows

- Set of states $\tilde{\mathcal{S}}$ is equal to the set of states that a chosen forward-search algorithm can attain from initial state s_0. For example, for GBFS, this corresponds to the search tree, open list, and closed list.
- Set of actions $\tilde{\mathcal{A}}$ available at a state $T \in \tilde{\mathcal{S}}$ is the set of nodes in the open list that can be expanded. This means that an action in $\tilde{\mathcal{M}}$ corresponds to expanding a leaf node.[4]
- Transition function $\tilde{\delta} \colon \tilde{\mathcal{S}} \times \tilde{\mathcal{A}} \to \tilde{\mathcal{S}}$ corresponds to the expansion of a leaf in GBFS. Assuming a leaf node s, which is a state $s \in \mathcal{S}$, we generate all children using all actions applicable in s, resolve duplicates in the search-tree, and append them to s in the search-tree structure.
- The reward function in Tree-MDP $\tilde{R} \colon \tilde{\mathcal{S}} \times \tilde{\mathcal{A}} \to \mathbb{R}$ rewards finding a state $s \in \mathcal{S}$ improving the function o. Formally the reward is defined as $\tilde{R}(T, \tilde{a}) = \max_{s \in \tilde{\delta}(T, \tilde{a})} o(s) - \max_{s \in T} o(s)$. A slightly abused notation $s \in T$ means all states within the search-tree T.

Tree-MDP is a valid MDP that can be solved by RL methods, since (i) the Markov property is satisfied and (ii) the reward is maximized when the state maximizing the objective function o is reached.

The above definition of Tree-MDP is general in the sense that the value, Q-functions, and policy are functions of an entire state (usually search-tree and an open list). But this makes them (i) incompatible with most implementations of GBFS methods, as they assume that the heuristic function h is a function only of a state $s \in \mathcal{S}$, and (ii) computationally expensive. The rest of this section therefore, discusses formulation, which makes the parametrization of policy functions compatible with GBFS methods. While this might decrease the performance, it enables exploring the effect of RL formulation without introducing additional sources of variance like heuristic functions with different powers. We leave this investigation to future work.

Terminology disambiguation. Sometimes, we need to make a clear distinction if we talk about MDP defining the problem $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \delta, R)$ or Tree-MDP $\tilde{\mathcal{M}} = (\tilde{\mathcal{S}}, \tilde{\mathcal{A}}, \tilde{\delta}, \tilde{R})$ defined in the previous section. In these cases, we will call the former Linear-MDP, since the search structure is linear, and the latter Tree-MDP, since the search structure is a tree. States of Linear-MDP (or MDP in general) are always denoted as s, whereas those of Tree-MDP are as T. When we discuss properties of MDP in general, which hold for any MDP, we will use the usual notation $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \delta, R)$.

Policy function. In order for the policy optimized by Tree-MDP to be compatible with GBFS solvers, we define the policy at state T as

$$\pi(\tilde{a} = s|T) = \frac{e^{-h(s)}}{\sum_{s' \in \mathcal{O}(T)} e^{-h(s')}}, \tag{3}$$

where $\mathcal{O}(T)$ is an open list of state T of Tree-MDP and action $\tilde{a}$ corresponds to expanding state $s \in \mathcal{O}(T)$. In every iteration of deterministic GBFS, the algorithm expands state

[2] Since the transition function is deterministic, actions a_i satisfying $s_{i+1} = \delta(s_i, a_i)$ are implicitly determined. Moreover, although there can be multiple actions leading from s_i to s_{i+1}, this is not problematic as the reward for all these actions is the same and is fully determined by s_i and s_{i+1}.

[3] This happens due to value / Q-function or policy not being optimal and / or stochasticity in rollouts.

[4] Actions of $\tilde{\mathcal{M}}$ therefore directly correspond to the states of $\mathcal{M}$.

s from the open list with minimal value of the heuristic function, which corresponds to a state with maximum probability computed according to (3). Since the function h does not depend on the state of Tree-MDP, it can be used as a heuristic in GBFS solvers. Section 3.1 discusses the relation of the policy defined in (3) to the recently proposed $\mathrm{L}_{\mathrm{gbfs}}$ loss.

We again emphasize that more general definitions of policy functions are possible, but they are not investigated in this paper for the reasons explained above.

3.1 Relation of Tree-MDP to Ranking Losses

$\mathrm{L}_{\mathrm{gbfs}}$ and L^* losses [Chrestien *et al.*, 2023; Piepenbrock *et al.*, 2022; Hao *et al.*, 2024] are loss functions for imitation learning, optimizing efficiency of the GBFS and A* search. The idea behind it is that the state in the open list, which lies on the example trajectory $\xi = (s_0, \ldots, s_n)$ should have the highest priority, which is translated to loss

$$\mathrm{L}_{\mathrm{gbfs}}(h, \xi) = \sum_{i=1}^{n} \sum_{s' \in \mathcal{O}(T_i)} \ell\big(h_\theta(s_i) - h_\theta(s')\big) \tag{4}$$

where $\mathcal{O}(T_i)$ is an open list of the GBFS after states $(s_0, s_1, \ldots, s_{i-1})$ have been expanded and ℓ is a suitable convex surrogate of a step function, for example, logistic loss $\ell = \log\big(1 + \exp(x)\big)$. The important property for efficiency of the search is that the state s_i has the smallest heuristic value among states in $\mathcal{O}(T_i)$, which can be similarly achieved by replacing the inner sum with a "logsoftmax" function as

$$\mathrm{L}_{\mathrm{softmax}}(h, \xi) = -\sum_{i=1}^{n} \log\left(\frac{e^{-h(s_i)}}{\sum_{s' \in \mathcal{O}(T_i)} e^{-h(s')}}\right). \tag{5}$$

The minus in front of h is there to comply with the convention that when optimizing a heuristic in GBFS, the smaller is better [5]. The term inside the sum in (5) now corresponds to the definition of policy $\pi(\tilde{a}|T)$ of a Tree-MDP in Equation (3).

According to results in [Chrestien *et al.*, 2023; Piepenbrock *et al.*, 2022; Hao *et al.*, 2024; Horčík *et al.*, 2025], heuristic functions optimized to rank almost always deliver better results than methods estimating cost-to-goal, which can now be viewed in the context of reinforcement learning, where policy methods often work better than value-based methods [Schulman *et al.*, 2017].

4 Reward

This section discusses the challenges in defining the reward function for problems that optimize the objective function o. The goal is to design a reward that orders trajectories according to the "natural order."

Definition 1 (Natural order)**.** *The reward function $G(\xi)$ satisfies the natural order property if for any two trajectories $\xi' = (s'_0, s'_1, s'_2, \ldots, s'_{n'})$ and $\xi'' = (s''_0, s''_1, s''_2, \ldots, s''_{n''})$ with the same initial state $s'_0 = s''_0$ and with respective total rewards $G(\xi')$ and $G(\xi'')$, the following two conditions are satisfied.*

[5] Interesting property of this change is that the complexity of the loss is now linear instead of quadratic, as criticized in [Hao *et al.*, 2024].

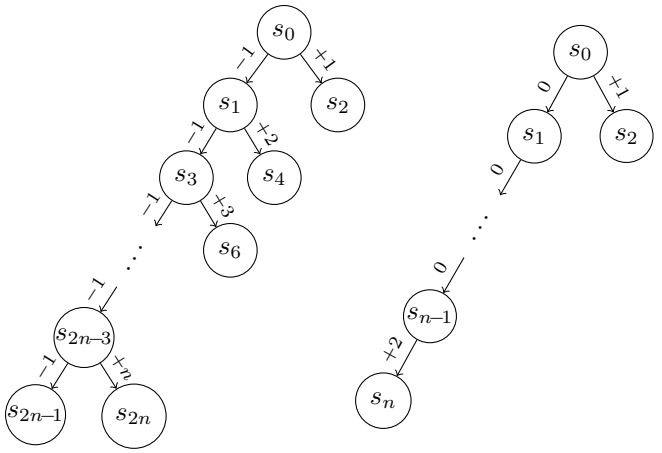

Figure 1: Two example MDPs. The numbers above each transition indicate the immediate rewards associated with those transitions. In the left MDP, s_2, s_4, …, s_{2n} are goal states. In the right MDP, s_2 and s_n serve as goal states, with s_2 representing a local optimum and s_n the global optimum.

1. *If $o(s'_{n'}) > o(s''_{n''})$, then $G(\xi') > G(\xi'')$. This requirement states that a better solution should always be preferred, regardless of the number of steps taken to find it.*
2. *If $o(s'_{n'}) = o(s''_{n''})$ and $n' < n''$, then $G(\xi') > G(\xi'')$. This requirement states that if both trajectories find a solution of equal quality, the one reached in fewer steps should be preferred. If $n' = n''$, then any ordering is acceptable.*

While the first condition is essential, the second condition formulates the desire to find solutions quickly and not wait until the timeout (used in the training). Below, the difficulties in formalizing the natural order are discussed.

4.1 Formalizing Reward for Reinforcement Learning

A classical approach [Chen and Tian, 2019] for the class of problems considered here is to define the reward as $R(s, a) = o(\delta(s, a)) - o(s)$, which can be used for both MDP and Tree-MDP definitions. But this definition suffers from the following issues.

Case $\gamma = 1$. For a discount factor $\gamma = 1$, it holds that $G(\xi) = \sum_{i=0}^{n-1} \big(o(s_{i+1}) - o(s_i)\big) = o(s_n) - o(s_0)$, which captures the goal of maximizing $o(s_n)$, but the length of the solution is not penalized. For problems where there exist multiple paths to the goal, this can lead to behavior where the solver can suggest much longer solutions when shorter ones exist. In fact, in pathological situations illustrated on the left side of Figure 1, the optimal heuristic might be non-informative, because all paths might seem equally good (leading to the same reward). $\gamma = 1$ is frequently used in episodic reinforcement learning and corresponds to solving the problem within a time limit without extra reward for solving the problem quickly, which is not very practical.

Case $\gamma < 1$. Prioritizing the shortest path is typically solved by setting the discount factor $\gamma < 1$. With the right set-

ting, the reward function $G(\xi) = \sum_{i=0}^{n-1} \gamma^i (o(s_{i+1}) - o(s_i))$ satisfies the natural order property, but with γ too small, it can introduce local minima into the problem, as illustrated on the right side of Figure 1. When deciding which action to take in state s_0, the reward for going right is 1 whereas the reward for going left is $2\gamma^{n-1}$. Therefore if $n > 1 - \frac{\log(2)}{\log(\gamma)}$, the agent will take the right action first. If the transition system contains dead ends, the agent gets stuck in local minima. Fortunately, the GBFS does not have dead ends, as these are resolved using open lists, and these local minima "just" make the time to find optima longer. But the reward function violates the conditions of the natural order property.

4.2 Optimizing Policy without Explicit Reward

An alternative approach to optimizing the policy is not to formulate the reward function at all, but to assume there exists an absolute ordering of trajectories. The idea is to sample multiple trajectories, select the best trajectory, and optimize the policy to prioritize the best trajectory.

Formally, assume we have m trajectories $\{\xi_i\}_{i=1}^m$ and let $\xi^\star = (s_0^\star, s_1^\star, \ldots, s_n^\star)$ be the best trajectory according to chosen ordering. This defines an empirical policy on the visited states: $\pi^\star(a|s) = 1$ if $s = s_i^\star$ for some $i \in \{0, \ldots, n-1\}$ and $\delta(s_i^\star, a) = s_{i+1}^\star$ for exactly one action a, and zero otherwise.[6] The current policy π_θ can be improved to prioritize $\pi^\star$ by minimizing KL-divergence $\mathrm{KL}(\pi^\star \| \pi_\theta)$.

Sampling multiple trajectories can be computationally expensive,[7] but it is cheap to obtain $\xi^\star$ in Tree-MDP, because the shortest path to the best node can be used instead. The rest discuss two different methods to use this method to optimize the policy of Tree-MDP. We emphasize that the rest of this section will use the notation of Tree-MDP.

Bootstrapping with $\mathrm{L}_{\mathbf{gbfs}}$ Loss

Let the best trajectory in the current search tree be $\xi^\star$, corresponding to expanding states $(s_0^\star, s_1^\star, \ldots, s_n^\star)$. In this case, minimizing the KL divergence equals

$$\mathrm{KL}(\pi^\star \| \pi_\theta) = \sum_{i=1}^{n} \sum_{\tilde{a} \in \mathcal{O}(T_i)} \pi^\star(\tilde{a}|T_i) \log \frac{\pi^\star(\tilde{a}|T_i)}{\pi_\theta(\tilde{a}|T_i)} \tag{6}$$

where $\mathcal{O}(T_i)$ is again the open list after expanding states $(s_0^\star, s_1^\star, \ldots, s_{i-1}^\star)$. Recognizing that empirical policy $\pi^\star$ is zero everywhere except when action $\tilde{a}$ corresponds to expanding state $s_i^\star$ in state T_i, the above can be rewritten as

$$-\sum_{i=1}^{n} \log(\pi_\theta(\tilde{a} = s_i^\star|T_i)) = -\sum_{i=1}^{n} \log \frac{e^{-h(s_i^\star)}}{\sum_{s' \in \mathcal{O}(T_i)} e^{-h(s')}}, \tag{7}$$

where we have substituted the formulation of policy compatible with GBFS. Equation (7) recovers the $\mathrm{L}_{\mathrm{gbfs}}$ loss with log-softmax loss function as defined in Equation (5). This formalization is effectively equal to a bootstrapping protocol [Arfaee *et al.*, 2011] optimizing the heuristic function by $\mathrm{L}_{\mathrm{gbfs}}$.

[6]The support of $\pi^\star$ equals to states in trajectory $\xi^\star$, which as seen shortly is sufficient.

[7]This approach of multiple rollout is used in currently popular group-relative policy optimization [Shao *et al.*, 2024].

Inspiration from Reinforcement Learning

The above loss function (7) might be off policy, because the best trace $\xi^\star$ might be very different from the sequence of states expanded by GBFS. We therefore propose to define a policy for every state of Tree-MDP observed during the search.

Let $(\bar{s}_0, \bar{s}_1, \ldots, \bar{s}_{\bar{n}})$ denote the sequence of states expanded by GBFS creating a sequence of Tree-MDP states $(\bar{T}_1, \bar{T}_2, \ldots, \bar{T}_{\bar{n}+1})$ with corresponding open lists. Furthermore, let $\xi^\star$ again be the shortest trajectory to the best state corresponding to expanding states $(s_0^\star, s_1^\star, \ldots, s_n^\star)$. The best known action at Tree-MDP state $\bar{T}_i$ is to expand the state from the open list closest to $s_n^\star$,

$$\bar{s}_{\bar{T}_i}^\star = \operatorname*{argmax}_{s_j^\star \in \mathcal{O}(\bar{T}_i)} j, \tag{8}$$

which defines the corresponding empirical policy as above. The loss function for minimizing the KL divergence can be simplified as above to

$$-\sum_{i=1}^{\bar{n}} \log(\pi_\theta(\tilde{a} = \bar{s}_{\bar{T}_i}^\star|\bar{T}_i)) = -\sum_{i=1}^{\bar{n}} \log \frac{e^{-h(\bar{s}_{\bar{T}_i}^\star)}}{\sum_{s' \in \mathcal{O}(\bar{T}_i)} e^{-h(s')}}. \tag{9}$$

This last formulation combines ideas from symbolic planning (extraction of the best trajectory of the solver) and reinforcement learning (using a sequence of states expanded during GBFS). An important feature of this formulation missing in (7), is that if an important state is frequently not expanded, it receives higher priority because it appears in multiple open lists. The importance of states is therefore automatically adjusted. Below, this loss is called *policy-optimizing* loss denoted as L_{po}.

5 Related Work

[Orseau *et al.*, 2018] derives a heuristic for GBFS from a policy network for the Linear-MDP, such that the maximum number of steps before finding a solution can be bounded. The policy network is trained using A3C while solving the Linear-MDP without considering the tree search. A subsequent work [Orseau and Lelis, 2021] uses the Bootstrap process [Arfaee *et al.*, 2011], removing the need for a trained policy network. The main difference to our work is that (i) our approach optimizes heuristic functions for Tree-MDP and not policy networks of Linear-MDP, (ii) our formalism can use off-the-shelf RL methods, and (iii) we can use all nodes in the search-tree in minibatch to optimize the heuristic.

As already discussed, [Chrestien *et al.*, 2023; Piepenbrock *et al.*, 2022; Hao *et al.*, 2024] optimize the heuristic function for GBFS and A* to rank states on good trajectories over other states. This work exposes the relation to policy gradient of a (Tree-MDP) (see Section 3.1) and demonstrates that off-the-shelf RL methods can optimize heuristic functions without examples of solved problems.

[Bernhard *et al.*, 2018] proposed to estimate cost-to-goal from Q-values by computing the reward in the original formulation of the planning problem. The setting is similar to [Orseau *et al.*, 2018], as the neural network estimating Q-values does not consider the dynamics of GBFS. Recall that

cost-to-goal was empirically shown in [Chrestien *et al.*, 2023; Horčík *et al.*, 2025] to be inferior to ranking-based methods.

[Bhardwaj *et al.*, 2017] formulates the problem similarly to the proposed Tree-MDP to minimize the cost of the search. But contrary to our approach, it trains policies by imitating oracles with full knowledge of the environment, providing optimal plans (this work does not use any oracle).

Representation of planning problems and related architectures of neural networks for heuristic functions is an active research area [Horčík and Šír, 2024; Toyer *et al.*, 2020; Chen *et al.*, 2024; Ferber *et al.*, 2020; Ståhlberg *et al.*, 2022]. This work is perpendicular, as the interest here is in how to optimize the heuristic function. But it builds on this research, as in the experimental section, we use the ObjectBinary architecture with parameters provided in [Horčík *et al.*, 2025].

The computation of policy and value functions in RL should not have static complexity and should be allowed a higher computational budget in difficult situations. [Farquhar *et al.*, 2017] proposes to construct a small tree from which the Q-value of a state is computed, which allows it to consider different options. Unlike this work, the tree is not used for the search, but to improve the precision of the estimate, which is a different problem from ours.

MCTSNets [Guez *et al.*, 2018] generalizes Monte-Carlo Tree Search (MCTS) by replacing the update rules by trainable neural networks with the goal of faster solving of planning problems (tested only on Sokoban). The solved problem is therefore the same as in this work. The difference is that here, we are interested in GBFS and particularly in the connection to RL and the possibility of using RL methods to optimize heuristic functions for GBFS. AlphaZero [Silver *et al.*, 2016] also optimizes MCTS and is designed for games rather than for planning, although it has been used in planning [Feng *et al.*, 2020]. Since the interest here is GBFS, we leave the proper comparison to future work.

6 Experiments

6.1 Compared Algorithms

We experimentally compare the following loss functions / methods.

$\mathrm{L_{gbfs}}$ with inequality constraints as proposed in [Chrestien *et al.*, 2023]. Since $\mathrm{L_{gbfs}}$ was designed for imitation learning, it was adapted to the bootstrap protocol by learning to imitate the shortest trajectory to the best node in the search tree (with the highest objective function).

To observe the effect of replacing inequalities in $\mathrm{L_{gbfs}}$ by `softmax` function, we tested the variant as defined in Equation (5) (and equally (7)) further denoted as $\mathrm{L_{gbfs}}$(softmax). As explained in Section 3.1, $\mathrm{L_{gbfs}}$ with softmax can be viewed as optimizing a policy that selects which states to expand from an open list.

We also evaluate Proximal Policy Optimization [Schulman *et al.*, 2017] (PPO) with the Linear-MDP (denoted as Linear-PPO) and with the Tree-MDP formulation taking into account search (denoted as Tree-PPO). To be comparable to other algorithms, the policy is implemented according to Equation (3). The additional benefit of this formulation of policy is that it naturally handles a varying number of actions.

Finally, the Levin Tree Search (denoted as LevinTS) algorithm proposed in [Orseau *et al.*, 2018] was adapted to the Bootstrap protocol as above by training the policy network on the shortest trajectory to the best node. We have not implemented [Orseau and Lelis, 2021] as it was not clear how to extend it to problems with a non-binary reward function.

6.2 Settings

The experimental protocol follows the bootstrap process [Arfaee *et al.*, 2011]. Models start with randomly initialized weights using a random number generator fixed to the same seed (1,2,3 for 3 repetitions of each experiment). For a given domain, the algorithm sequentially iterates over available problem instances and for each problem instance performs the following steps:

1. run the search algorithm using the current model until the problem is solved or the maximum time limit of 30s is exhausted,
2. extract the training samples from the search algorithm,
3. update the model parameters using the collected samples.

All evaluated algorithms except Linear-PPO used greedy best-first search as described above in Section 2, with the change that the `dequeue!` operation in line 8 randomly samples a state from the policy introduced to promote exploration of the search space. Linear-PPO samples the next state from policy until a terminal state is reached or a predefined cutoff is exceeded. Unlike GBFS, the result is a linear trajectory. The domains and problem instances were taken from IPC learning track challenge [Taitler *et al.*, 2024]. The best trajectory is not copied across epochs.

The objective function is equal to the size of the intersection of atoms in a state with atoms in a goal state. This provides a denser reward function, and finding the maximum of this objective function means the problem was solved. The reward function needed by the MDP formulations used by Linear-PPO and Tree-PPO was as defined earlier in Section 2 and Section 3 with $\gamma = 0.99$.

Policy, heuristic, and value functions are parameterized using neural networks realized as ObjectBinary encoding with features on edges, as described in [Horčík *et al.*, 2025]. This architecture was chosen because it is among the best on problems from IPC 2023 challenge. The neural networks used a single message-passing layer with a hidden output dimension 32, followed by `sum` and `max` pooling functions, followed by a two-layer network with a hidden dimension 32 and output dimension 1 with ReLU nonlinearities. ADAM optimizer was used with learning rate $\{10^{-3}, 5 \cdot 10^{-4}, 10^{-4}\}$ treated as a hyper-parameter. The maximum number of Epochs was set to 10, except for Linear-PPO, where we have experimented with the following combinations of maximum epochs / maximum simulation time $\{(10/30), (20/15), (30/10), (60/5)\}$. The last hyper-parameter was the number of gradient descent steps after each simulation, where $\{1, 4, 16\}$ steps were tested. The best combination of hyperparameters was selected on the validation set, and the reported values are on the test set.

	L_{gbfs}	L_{gbfs}(softmax)	L_{po}	Tree-PPO	Linear-PPO	LevinTS
blocksworld	84.1 ± 0.0	$\mathbf{84.5 \pm 0.3}$	83.8 ± 1.1	64.7 ± 1.7	56.8 ± 6.1	32.8 ± 1.8
ferry	$\mathbf{84.1 \pm 0.0}$	$\mathbf{84.1 \pm 0.0}$	$\mathbf{84.1 \pm 0.0}$	$\mathbf{84.1 \pm 0.0}$	14.8 ± 6.9	18.3 ± 0.3
floortile	11.9 ± 1.7	16.4 ± 0.8	$\mathbf{53.3 \pm 5.8}$	8.6 ± 0.0	1.6 ± 0.5	8.6 ± 0.0
miconic	$\mathbf{68.8 \pm 0.0}$	$\mathbf{68.8 \pm 0.0}$	$\mathbf{68.8 \pm 0.0}$	$\mathbf{68.8 \pm 0.0}$	18.2 ± 5.9	40.7 ± 0.6
rovers	68.8 ± 0.0	68.8 ± 0.0	$\mathbf{69.0 \pm 0.3}$	65.1 ± 2.4	62.4 ± 0.9	12.0 ± 1.2
satellite	73.5 ± 0.0	$\mathbf{73.7 \pm 0.3}$	72.7 ± 0.8	69.3 ± 0.5	66.7 ± 1.9	12.9 ± 0.6
sokoban	64.0 ± 0.6	$\mathbf{64.2 \pm 0.3}$	64.0 ± 0.6	61.9 ± 1.4	8.5 ± 0.9	41.6 ± 1.4
spanner	77.8 ± 0.3	77.6 ± 0.0	77.6 ± 0.0	77.6 ± 0.6	$\mathbf{80.1 \pm 0.3}$	66.5 ± 0.0
transport	74.2 ± 0.8	74.3 ± 0.3	$\mathbf{74.4 \pm 0.3}$	66.5 ± 3.9	47.8 ± 33.3	8.5 ± 0.0
total	67.5 ± 0.1	68.0 ± 0.1	$\mathbf{72.0 \pm 0.8}$	63.0 ± 1.0	39.6 ± 4.5	26.9 ± 0.2

Table 1: Average fraction of solved problems (in percents) and standard deviation of solved problems in 10 epochs on domains from IPC 2023 learning track challenge. The average and standard deviation are computed from three repetitions of the experiment. Liner-PPO and LevinTS are baseline algorithms, L_{gbfs} is a loss previously proposed for imitation learning and here adapted for Bootstrap protocol. L_{gbfs}(softmax) and importantly L_{po} are proposed here.

The experiments were implemented in the Julia 1.11.7 language built on top of `PDDL.jl` and `SymbolicPlanners.jl` [Zhi-Xuan, 2022]. The implementation of ObjectBinary encoding of states was taken from the official repository of [Horčík *et al.*, 2025]. All experiments were evaluated on Intel Xeon Gold 6146 using 8 cores and 32GB of memory limit allocated by the Slurm scheduler, with a maximum running time of 72 hours. The code will be made public upon acceptance.

6.3 Experimental Results

Table 1 shows the fraction of solved problems (in percents) after 10 epochs. The first five columns are organized such that they transition from planning formalism to RL formalism, the proposed L_{po} sits in the middle. LevinTS is treated separately.

Tree-PPO performs much better than Linear-PPO, which should not be surprising, as methods utilizing tree search are generally more powerful due to the advantage of the search, and PPO optimizes the efficiency of the search directly.

All methods derived from L_{gbfs} work similarly well. We observe that the softmax version of L_{gbfs} is mildly better, but the small differences are likely just noise. The proposed L_{po} performs overall the best (it is the only method solving floortile with non-trivial accuracy), and importantly, where it is worse than the best, it is by less than one percent. Therefore, the performance is stable across all domains. We attribute this to the inheritance of the best from planning and RL worlds. Using the structure of the search tree, it extracts the best trajectory to form a strong learning signal. Simultaneously, it creates the minibatch such that it improves / corrects every step of GBFS, emphasizing parts where the current policy is making costly mistakes.

Tree-PPO is worse than L_{gbfs} losses, which we believe is due to the need to train the value network in order to train the policy.

LevinTS was, to our surprise, inferior to Tree-PPO and to L_{gbfs} losses. Our investigation suggests this is caused by a combination of LevinTS being conservative (it penalizes depth, forcing the search to be more breadth-first search) and the parametrization of policy optimized in Linear-MDP formalism. Furthermore, all variants of L_{gbfs} and Tree-PPO directly optimize the efficiency of the search.

7 Conclusion

This paper has bridged the gap between learning heuristic functions for greedy best-first search (GBFS) and reinforcement learning (RL). The proposed Tree-MDP formulation combines the dynamics of GBFS with the dynamics of the solved problem, which enables principled learning of heuristic functions by reinforcement learning methods. Experimental comparison using the Proximal Policy Optimization method confirmed that using the Tree-MDP formulation delivers markedly better results.

Policy learning in the Tree-MDP formalism was then connected to recently proposed ranking losses for optimization of heuristic functions. This connection facilitated the design of a new loss function, called policy loss, combining strengths from both planning and reinforcement learning. In the experimental comparison, the policy loss delivered overall the best results, improving the established prior art by 30 percents. An interesting advantage of policy loss is that it does not require an explicitly defined reward function, as it suffices to define a total ordering on the space of trajectories, and as we have discussed, defining the reward function can be difficult.

We hope that this formalism will facilitate the transfer of ideas between planning and reinforcement learning, but also help with the identification of differences, as we have shown that using the search tree structure can lead to better algorithms. In the future, we would like to investigate the relation of Tree-MDP to Monte-Carlo tree search and methods that compute policy from an entire search tree. Furthermore, we would like to make the training computationally cheaper by investigating whether all states in the search tree are needed in the minibatch.

References

[Anderson *et al.*, 2018] Hyrum S. Anderson, Anant Kharkar, Bobby Filar, David Evans, and Phil Roth. Learning to evade static PE machine learning malware models via reinforcement learning. *CoRR*, abs/1801.08917, 2018.

[Arfaee *et al.*, 2011] Shahab Jabbari Arfaee, Sandra Zilles, and Robert C. Holte. Learning heuristic functions for large state spaces. *Artif. Intell.*, 175(16-17):2075–2098, 2011.

[Bernhard *et al.*, 2018] Julian Bernhard, Robert Gieselmann, Klemens Esterle, and Alois Knoll. Experience-based heuristic search: Robust motion planning with deep q-learning. In *2018 21st international conference on intelligent transportation systems (itsc)*, pages 3175–3182. IEEE, 2018.

[Bhardwaj *et al.*, 2017] Mohak Bhardwaj, Sanjiban Choudhury, and Sebastian Scherer. Learning heuristic search via imitation. In *Conference on Robot Learning*, pages 271–280. PMLR, 2017.

[Chen and Tian, 2019] Xinyun Chen and Yuandong Tian. Learning to perform local rewriting for combinatorial optimization. *Advances in neural information processing systems*, 32, 2019.

[Chen *et al.*, 2024] Dillon Z Chen, Felipe Trevizan, and Sylvie Thiébaux. Return to tradition: Learning reliable heuristics with classical machine learning. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 34, pages 68–76, 2024.

[Chrestien *et al.*, 2023] Leah Chrestien, Stefan Edelkamp, Antonin Komenda, and Tomas Pevny. Optimize planning heuristics to rank, not to estimate cost-to-goal. *Advances in Neural Information Processing Systems*, 36:25508–25527, 2023.

[Farquhar *et al.*, 2017] Gregory Farquhar, Tim Rocktäschel, Maximilian Igl, and Shimon Whiteson. Treeqn and atreec: Differentiable tree-structured models for deep reinforcement learning. *arXiv preprint arXiv:1710.11417*, 2017.

[Feng *et al.*, 2020] Dieqiao Feng, Carla P Gomes, and Bart Selman. A novel automated curriculum strategy to solve hard sokoban planning instances. *Advances in Neural Information Processing Systems*, 33:3141–3152, 2020.

[Ferber *et al.*, 2020] Patrick Ferber, Malte Helmert, and Jörg Hoffmann. Neural network heuristics for classical planning: A study of hyperparameter space. In *ECAI 2020*, pages 2346–2353. IOS Press, 2020.

[Franco *et al.*, 2017] Santiago Franco, Álvaro Torralba, Levi HS Lelis, and Mike Barley. On creating complementary pattern databases. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence*, pages 4302–4309, 2017.

[Groshev *et al.*, 2018] Edward Groshev, Maxwell Goldstein, Aviv Tamar, Siddharth Srivastava, and Pieter Abbeel. Learning generalized reactive policies using deep neural networks. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 28, pages 408–416, 2018.

[Guez *et al.*, 2018] Arthur Guez, Théophane Weber, Ioannis Antonoglou, Karen Simonyan, Oriol Vinyals, Daan Wierstra, Rémi Munos, and David Silver. Learning to search with mctsnets. In *International conference on machine learning*, pages 1822–1831. PMLR, 2018.

[Hao *et al.*, 2024] Mingyu Hao, Felipe Trevizan, Sylvie Thiébaux, Parick Ferber, and Jörg Hoffmann. Learned pairwise rankings for greedy best-first search. In *Proc. ICAPS Workshop on Reliable Data-Driven Planning and Scheduling*, 2024.

[Hart *et al.*, 1968] Peter E Hart, Nils J Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.

[Horčík and Šír, 2024] Rostislav Horčík and Gustav Šír. Expressiveness of graph neural networks in planning domains. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 34, pages 281–289, 2024.

[Horčík *et al.*, 2025] Rostislav Horčík, Gustav Šír, Vítězslav Šimek, and Tomáš Pevnỳ. State encodings for gnn-based lifted planners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 26525–26533, 2025.

[Katz *et al.*, 2018] Michael Katz, Shirin Sohrabi, Horst Samulowitz, and Silvan Sievers. Delfi: Online planner selection for cost-optimal planning. *IPC-9 planner abstracts*, pages 57–64, 2018.

[Korf, 1985] Richard E Korf. Depth-first iterative-deepening: An optimal admissible tree search. *Artificial intelligence*, 27(1):97–109, 1985.

[Moraru *et al.*, 2019] Ionut Moraru, Stefan Edelkamp, Santiago Franco, and Moises Martinez. Simplifying automated pattern selection for planning with symbolic pattern databases. In *Joint German/Austrian Conference on Artificial Intelligence (Künstliche Intelligenz)*, pages 249–263. Springer, 2019.

[Nilsson, 1971] Nils J Nilsson. Problem-solving methods in artificial intelligence. *Artificial Intelligence*, 5, 1971.

[Orseau and Lelis, 2021] Laurent Orseau and Levi HS Lelis. Policy-guided heuristic search with guarantees. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 12382–12390, 2021.

[Orseau *et al.*, 2018] Laurent Orseau, Levi Lelis, Tor Lattimore, and Théophane Weber. Single-agent policy tree search with guarantees. *Advances in Neural Information Processing Systems*, 31, 2018.

[Piepenbrock *et al.*, 2022] Jelle Piepenbrock, Tom Heskes, Mikoláš Janota, and Josef Urban. Guiding an automated theorem prover with neural rewriting. In *International Joint Conference on Automated Reasoning*, pages 597–617. Springer International Publishing Cham, 2022.

[Schulman *et al.*, 2017] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[Shao *et al.*, 2024] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang,

Mingchuan Zhang, YK Li, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[Silver *et al.*, 2016] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *nature*, 529(7587):484–489, 2016.

[Silver *et al.*, 2021] Tom Silver, Rohan Chitnis, Joshua Tenenbaum, Leslie Pack Kaelbling, and Tomás Lozano-Pérez. Learning symbolic operators for task and motion planning. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 3182–3189. IEEE, 2021.

[Ståhlberg *et al.*, 2022] Simon Ståhlberg, Blai Bonet, and Hector Geffner. Learning general optimal policies with graph neural networks: Expressive power, transparency, and limits. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 32, pages 629–637, 2022.

[Sutton *et al.*, 1998] Richard S Sutton, Andrew G Barto, et al. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.

[Taitler *et al.*, 2024] Ayal Taitler, Ron Alford, Joan Espasa, Gregor Behnke, Daniel Fišer, Michael Gimelfarb, Florian Pommerening, Scott Sanner, Enrico Scala, Dominik Schreiber, Javier Segovia-Aguas, and Jendrik Seipp. The 2023 International Planning Competition. *AI Magazine*, 45(2):280–296, 2024.

[Toyer *et al.*, 2020] Sam Toyer, Sylvie Thiébaux, Felipe Trevizan, and Lexing Xie. Asnets: Deep learning for generalised planning. *Journal of Artificial Intelligence Research*, 68:1–68, 2020.

[Zhi-Xuan, 2022] Tan Zhi-Xuan. *Pddl. jl: An extensible interpreter and compiler interface for fast and flexible ai planning*. PhD thesis, Massachusetts Institute of Technology, 2022.

[Zoph and Le, 2017] Barret Zoph and Quoc V. Le. Neural architecture search with reinforcement learning. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net, 2017.